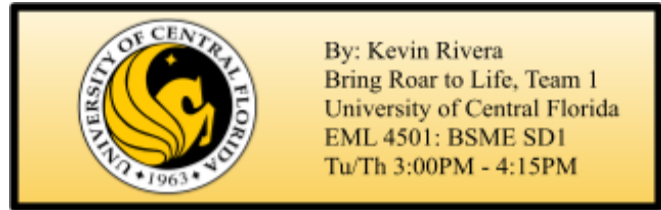


Technological Memo of Digital and Controls Systems



Introduction

Our team has been tasked with constructing a free standing quadrupedal robot able to achieve a walking motion with dinosaur anatomy. To accomplish this our quadrupedal robot will require the implementation of digital and control systems that will drive its ability to walk as well as any additional functions/features that have been approved and discussed by the team such as for example opening and closing the mouth of our robot etc. The Tech memo is about the selection and a catalog of options for digital and control systems. Below the following will be discussed: a summary of control theory with a theoretical analysis as pertains to our project, discussion of the Hardware such as microcontrollers, microcomputers, and microprocessors, the programing languages and operation systems with regards to use in robotics, a discussion of references for algorithmic and mathematical methods for use in controlled and autonomous robotics systems. Achieving these objectives will allow our team to make more informed decisions when creating design requirements as well as create a reference document for future design and development.

Control Theory and Systems

Control theory “is a mathematical framework that gives us the tools to develop autonomous systems” [2]. A feed forward controller or open loop controller: takes in the desired output/reference and generates the control inputs without ever needing to measure the state. A feed forward controller does not rely on feedback from the state of the system which is why it is an open loop, it never loops back. Feedback control/closed loop control relies on both the reference and the current state of the system to generate the appropriate control inputs. This allows for if the state of the system begins to deviate due to errors or external disturbances the output is fed back into the feedback control so it can adjust the input accordingly. Refer to figure 1. for types of feedback controllers. State Estimation is

another critical part of control theory. This is because the state of the system is read using sensors, so measurement noise becomes a problem. We must also be able to observe every state of the system even if not every state is directly measured. So the goal is to “reduce measurement noise” and “manipulate measurements in such a way that allows us to accurately estimate the state of the system.” [2] It is also vital to test the control system via analysis, simulation, and test to ensure it works properly and gather all data from testing for final conclusions and edits. The primary requirement of our project is to achieve a walking motion, implementing control theory will aid in the design process. In order to achieve motion control, control theory can be used to create a model to regulate the position, speed, and torque of the robot. For example a feedback controller or closed loop controller would adjust the motor input based on the desired state and the actual measured state. Another example would be for trajectory tracking, the control system can in real time adjust the input based on disturbances.

Hardware

The purpose of this section is to define and create an understanding of the hardware and aims to give the reader understanding when selecting Hardware for project use. Proper selection is crucial for planning and implementation of the project. Determining the proper hardware for the project is crucial to achieve the locomotion goal, factors like processor architecture, clockspeed, RAM, Flash memory, I/O ports, power consumption, operating voltage, and the development environment, must be taken into consideration.

The architecture of a processor is a direct determinant of the performance of the system as “microcontrollers are generally classified according to processor characteristics such as processor architecture, processor word processing length, processor clock operating frequency, and so on.” This determines how well it handles data. For example, “RISC architectural microprocessors are generally

preferred in electronic applications such as control and automation, [while] CISC architectures are used in microcomputer applications such as signal processing and embedded systems” [3]. Therefore when choosing the architecture it should account for computational power and the complexity of its intended application. The Clock speed, measured in MHz or GHz, showcases how fast a processor can carry out instructions, thus “attention should be paid to technical features such as word processing length, working clock frequency, memory amount, bus width and internal additional hardware”[3]. If it has a higher clock frequency then the general consciousness is that there is also an increase in power consumption and heat generation. Another key factor to take into account is memory. Random access memory or RAM “stores the information that the processor needs during its operation, while ROM, PROM, EPROM, and EEPROM memories...store information such as command set, program and program data”[3]. Enough Ram is important for dynamic operations while Flash memory is generally used to store firmware or any permanent code. Having enough Flash storage is a necessity so it can adequately hold all the required data and code. Just as important as the memory capacity, the I/O ports are equally as crucial to consider. It is recommended that the designer should list the hardware needed for different “ communication interfaces (USB, SPI, UART etc.)” and list the “port connections...for the input / output elements such as sensor, LCD display, driver and relay”[3]. How much power the system will consume is also important to consider especially if it is battery-powered or energy-sensitive. It is important to check the “ energy consumption, operating frequency and other variables” to assure you are making the correct choice in the micro controller [3]. When looking at operating voltage it is important to look at voltage compatibility since it can directly impact the ability and safety of a system. Finally it is important to check the “check the development environment and programming languages provided for the hardware...[and] search the programming interfaces and tools to be used” [3]. Together all these

factors are crucial to understand when trying to make the proper choice for which microcontroller to be used for our project.

Software/Programing

The purpose of this section is to create an understanding of the different types of software tools and programming languages. It aims to give the reader understanding when selecting software and programming language for project use. Selecting the appropriate software tools are crucial to insure performance, compatibility and development efficiency. Robotics has a diverse range of programming languages that it relies on. Each language has its own strengths in terms of control abstraction and usability. C/C++ are one of the most commonly used languages due to their low-level efficiency and hardware access in industrial and research robotics, which offered extensions for controlling sensory, actuators, and planning activities [4]. In comparison, Python is versatile for high level logic. It is popularly used for scripting and AI [4]. Robot-specific extensions for Java, like motion description languages have been developed emphasizing object-specific abstraction with portability [4]. Another programming language is MatLab, a specialized environment design for data analysis, algorithm development, and simulation [5]. ROS supports multiple languages, providing tools and libraries for robot software development making it flexible to work with [6]. Micropython is another lightweight and “efficient implementation of the Python 3 programming language that includes a small subset of the Python standard library and is optimised to run on microcontrollers” [7]. Lua is one of the lightweight scripting languages used for rapid prototyping and embedded scripting, it was “designed to be simple, small, portable, fast and easily embedded into applications” [8].

When selecting the programming language there are multiple factors to consider such as: the execution speed and real-time capabilities, hardware compatibility and low-level access, library and

API support, community support, and the integration with existing robotics platforms, as stated in figure 2. It is important to evaluate the programming language in relation to the intended task, performance constraints, hardware choices etc.

Methods for Algorithmic and Mathematical Techniques

The purpose of this section is to identify and summarize key methods and programming techniques commonly used in robotics applications. Below is a table (figure 3.) summarizing the technique, purpose, area of application and a source with the method being used. The table in figure 3. can be applicable to the making of our project for reference of possible techniques to use.

Conclusion

Further research into this subject is recommended once design requirements have been designated. The background knowledge as well as all the research provided in this document should be valuable reference for multiple of the development and design phases. I believe that the key concepts provided here should allow us to do the following: select our hardware, software, design the control system structure and architecture, and implement the techniques mentioned or variations as needed for our needs. It is recommended that further study also looks into the cost and difficulty of implantation and use, this Technology memo fails to take these factors into account and is primarily of the purpose of familiarizing the team with the methods, models, key concepts, and systems used in digital systems and control systems in a generalized matter.

APPENDIX

Figure 1. Types of feedback controllers

Type of Controller	Key Characteristics
Linear	Based on linear system assumptions; easy to analyze and design (e.g., PID, state feedback)
Nonlinear	Handles nonlinear dynamics; can deal with real-world complexities
Robust	Maintains performance under model uncertainties and disturbances
Adaptive	Adjusts its parameters in real-time based on system changes
Optimal	Minimizes a defined cost function (e.g., LQR, MPC)
Predictive	Uses a model to predict future behavior and optimize control actions (e.g., Model Predictive Control - MPC)
Intelligent	Uses AI techniques (e.g., fuzzy logic, neural networks, reinforcement learning)

Figure 2. Programming language selection table

Factor	Description
Execution speed and real-time capabilities	Critical for time-sensitive control tasks where delays can compromise performance.
Hardware compatibility and low-level access	Ability to interact directly with hardware components such as sensors, actuators, and controllers.
Library and API support (e.g., for sensors, motors, vision)	Availability of pre-built modules and frameworks to accelerate development in areas like sensing, motion, and perception.
Community support and documentation	Access to tutorials, forums, and technical references that reduce development time and troubleshooting difficulty.
Integration with existing robotics platforms	Ease of interoperability with frameworks like ROS and embedded systems, ensuring scalability and modularity.

Figure 3. Techniques used in Robotics

Techniques In Robotics			
Technique	Purpose	Application Area	Source / Reference
PID Controller	Cascaded PID for accurate foot-tip trajectory tracking on a quadruped leg	Quadruped leg servo control / trajectory tracking	[1]
LQR (Linear Quadratic Regulator)	Optimal feedback (Riccati/LQR) integrated with quadruped planning/control to improve robust motion tracking	Dynamic quadruped locomotion (trotting/pacing)	[11]
Kalman Filter (KF)	State estimation under Gaussian noise	General robotics state estimation	[12]
Extended Kalman Filter (EKF)	Classical Gaussian state estimation (derives and analyzes the KF)	General robotics state estimation	[12]
Particle Filter	Sequential Monte Carlo methods; practical tutorial and use for localization	General estimation + (see below PF legged)	[13]
Model Predictive Control (MPC)	Representation-Free MPC (RF-MPC): represents orientation with a rotation matrix, uses variation-based linearization and QP transcription; demonstrated real-time 250 Hz control and dynamic motions including a controlled backflip.	Quadruped robot dynamic locomotion and acrobatics (Panther platform).	[9]
A Algorithm*	Foundational text on graph/discrete planning incl. A* properties and use	General motion/discrete path planning	[14]
RRT / RRT*	Sampling-based motion planning	General motion planning	[15]
Deep Neural Networks (DNNs)	Perception, classification, control policies	Vision-based Navigation, Object Recognition	[18]
CNNs (Convolutional Neural Nets)	Feature extraction from images	Robotic Vision, SLAM	[17]
Reinforcement Learning (RL)	Learning control policies via rewards	Autonomous Navigation, Manipulation	[16]
Inverse Kinematics	Convert end-effector position to joint angles	Humanoid Robots, Manipulators	[27]
Forward Kinematics	Compute end-effector position from joint states	Robot Motion Modeling	[27]
SLAM (Graph-based, EKF, etc.)	Simultaneous localization and mapping	Mobile Robots, Autonomous Vehicles	[19]
Bezier and B-Spline Curves (Parametric Animation Curves)	Smooth trajectory generation through parametric curves	Motion planning for arms, mobile robots	[20]
Principal Component Analysis (PCA)	Dimensionality reduction and feature extraction	Sensor fusion, gesture recognition, object tracking	[21]
Trajectory Optimization (CHOMP)	Gradient-based trajectory optimization	Motion planning for manipulators and legged robots	[22]
Trajectory Optimization (STOMP)	Stochastic trajectory optimization with constraints	High-dimensional motion planning	[23]
Central Pattern Generators (CPGs)	Generate rhythmic signals for periodic movement (e.g., walking gaits)	Quadruped locomotion, bio-inspired robots	[24]
IMU Fusion (e.g., EKF)	Combine IMU data with other sensors (e.g., encoders, GPS) to estimate pose	Balance control, walking on uneven terrain	[25]
ROS Teleop / Joystick Input	Manual control of robot motion using joystick or keyboard commands	Manual operation, remote teleoperation	[6]
Direct Control Loops	Send low-level velocity or position commands directly to actuators	Low-latency control, gait tuning	[26]

REFERENCES

- [1] A. M. El-Dalatony, T. Attia, H. Ragheb, and A. M. Sharaf, "Cascaded PID Trajectory Tracking Control for Quadruped Robotic Leg," *International Journal of Mechanical Engineering and Robotics Research*, vol. 12, pp. 40–47, Jan. 2023, doi: <https://doi.org/10.18178/ijmerr.12.1.40-47>.
- [2] MATLAB, "Everything You Need to Know About Control Theory," *YouTube*, Oct. 27, 2022. https://www.youtube.com/watch?v=IBC1nEq0_nk&list=PLPajznaQQtnUmnhak6U10IQSNVDX4dGfp (accessed Sep. 20, 2025).
- [3] Y. Güven, E. Coşgun, S. Kocaoğlu, H. Gezici, and E. Yılmazlar, "Understanding the concept of microcontroller based systems to choose the best hardware for applications.," *123.216.36*, Jul. 2019, doi: <https://hdl.handle.net/20.500.11857/1024>.
- [4] G. Biggs and B. Macdonald, "A Survey of Robot Programming Systems." Accessed: Apr. 28, 2024. [Online]. Available: https://www.societyofrobots.com/robottheory/Survey_of_Robot_Programming_Systems.pdf
- [5] "Documentation - MATLAB & Simulink," *www.mathworks.com*. https://www.mathworks.com/help/?s_tid=mlh_sn_help
- [6] A. Koubaa, Ed., *Robot Operating System (ROS)*. Cham: Springer International Publishing, 2016. doi: <https://doi.org/10.1007/978-3-319-26054-9>.
- [7] MicroPython, "MicroPython - Python for microcontrollers," *micropython.org*. <https://micropython.org/>
- [8] R. Ierusalimsky, H. Luiz, De Figueiredo, and W. Celes, "The Evolution of Lua." Available: <https://www.lua.org/doc/hopl.pdf>

- [9] Y. Ding, Abhishek Pandala, C. Li, Y.-H. Shin, and Hae Won Park, “Representation-Free Model Predictive Control for Dynamic Motions in Quadrupeds,” *IEEE Transactions on Robotics*, vol. 37, no. 4, pp. 1154–1171, Aug. 2021, doi: <https://doi.org/10.1109/tro.2020.3046415>.
- [10] H. Meng, S. Zhang, W. Zhang, and Y. Ren, “Optimizing actual PID control for walking quadruped soft robots using genetic algorithms,” *Scientific Reports*, vol. 14, no. 1, Oct. 2024, doi: <https://doi.org/10.1038/s41598-024-77100-7>.
- [11] G. Xin *et al.*, “Robust Footstep Planning and LQR Control for Dynamic Quadrupedal Locomotion,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4488–4495, Mar. 2021, doi: <https://doi.org/10.1109/lra.2021.3068695>.
- [12] S. Thrun, W. Burgard, and D. Fox, *Probabilistic robotics*. Cambridge, Mass.: Mit Press, 2010.
- [13] J. Elfring, E. Torta, and R. van de Molengraft, “Particle Filters: A Hands-On Tutorial,” *Sensors*, vol. 21, no. 2, p. 438, Jan. 2021, doi: <https://doi.org/10.3390/s21020438>.
- [14] Steven Michael Lavalle, *Planning algorithms*. New York (Ny): Cambridge University Press, , Cop, 2014.
- [15] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, Jun. 2011, doi: <https://doi.org/10.1177/0278364911406761>.
- [16] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, Oct. 2020, doi: <https://doi.org/10.1126/scirobotics.abc5986>.
- [17] A. Villarreal *et al.*, “Fast and Continuous Foothold Adaptation for Dynamic Locomotion Through CNNs,” vol. 4, no. 2, pp. 2140–2147, Feb. 2019, doi: <https://doi.org/10.1109/lra.2019.2899434>.

- [18] J. Hwangbo *et al.*, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, p. eaau5872, Jan. 2019, doi: <https://doi.org/10.1126/scirobotics.aau5872>.
- [19] C. Cadena *et al.*, “Past, Present, and Future of Simultaneous Localization And Mapping: Towards the Robust-Perception Age,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016, doi: <https://doi.org/10.1109/TRO.2016.2624754>.
- [20]
- G. Duarte *et al.*, “Quadruped Robot Control: An Approach Using Body Planar Motion Control, Legs Impedance Control and Bézier Curves,” *Sensors*, vol. 24, no. 12, pp. 3825–3825, Jun. 2024, doi: <https://doi.org/10.3390/s24123825>.
- [21] I. T. Jolliffe and J. Cadima, “Principal component analysis: a review and recent developments,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, Apr. 2016, doi: <https://doi.org/10.1098/rsta.2015.0202>.
- [22] M. Zucker *et al.*, “CHOMP: Covariant Hamiltonian optimization for motion planning,” *The International Journal of Robotics Research*, vol. 32, no. 9–10, pp. 1164–1193, Aug. 2013, doi: <https://doi.org/10.1177/0278364913488805>.
- [23] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, “STOMP: Stochastic trajectory optimization for motion planning,” *2011 IEEE International Conference on Robotics and Automation*, May 2011, doi: <https://doi.org/10.1109/icra.2011.5980280>.
- [24] A. J. Ijspeert, “Central pattern generators for locomotion control in animals and robots: A review,” *Neural Networks*, vol. 21, no. 4, pp. 642–653, May 2008, doi: <https://doi.org/10.1016/j.neunet.2008.03.014>.
- [25] M. Bloesch *et al.*, “State Estimation for Legged Robots - Consistent Fusion of Leg Kinematics and,” Jul. 2012, doi: <https://doi.org/10.15607/rss.2012.viii.003>.

[26] F. Sygulla *et al.*, “An EtherCAT-Based Real-Time Control System Architecture for Humanoid Robots,” *Conference on Automation Science and Engineering*, Aug. 2018, doi: <https://doi.org/10.1109/coase.2018.8560532>.

[27] J. J. Craig, *Introduction to robotics : mechanics and control*. Upper Saddle Hall: Pearson Educacion Internacional, 2005.